AI-Based Food Recognition with Nutrition-Aware Recommendations

Jaspreet Singh*, Inderpal Singh†
*† Department of Computer Science and Engineering
National Institute of Technology Srinagar
Kashmir 190006, India
Email: [student email]
Project Guide: Dr. Lavanya Madhuri Bollipo
Project Co-Guide: Dr. Insha Majeed

***Abstract*—For dietary assessment, visual food labels must be tied to reliable nutrient records. This paper presents an AI-based Indian food recognition system that detects visible food items, maps detector classes to nutrition entries, computes macronutrients, and returns nutrition-aware recommendations from the computed values. The main technical contribution is a conservative nutrition-grounding layer between the YOLO detector and the recommendation module. The layer uses canonical class names, reviewed semantic mappings, source-backed supplemental rows, thresholded fallback matching, and provenance metadata rather than relying on unconstrained string similarity. Five YOLO-format Indian-food datasets were merged into a 72-class dataset with 48,693 exported image-label pairs. A YOLO11s detector trained on this dataset achieved 0.820 mAP@50 and 0.680 mAP@50:95 on the held-out test split. The nutrition database contains 1,010 food records and 103 detector-to-database mappings, covering all 72 expanded classes. Macro verification reported 51 verified mappings, 21 review-level mappings, and zero failed mappings. The prototype connects image upload, detection overlays, nutrition lookup, serving adjustment, and personalized recommendations while keeping nutrient arithmetic separate from recommendation text.**

***Index Terms*—food recognition, nutrition-aware recommendations, YOLO, Indian food dataset, dietary assessment, macronutrients**

SUBMISSION-BLOCKING AUDIT: SEPTEMBER 2026

This is a working draft, not a submission-ready manuscript. A read-only audit of all 48,693 exported images found 138 groups of byte-identical and decoded-pixel-identical images spanning splits (63 train–validation, 56 train–test, and 19 validation–test groups), affecting 276 images. Strict annotation validation also flagged 65 images, including zero-width boxes. Historical detector metrics below are retained for traceability, but do not establish clean held-out generalization. Source exports already include augmented variants, so training-only augmentation in the merged export does not establish independence. Dataset lineage, grouped splitting and independent evaluation must be resolved before submission.

Lookup coverage was 19/72 for normalized exact matching, 55/72 without precomputed mappings and 72/72 for the current mapping layer. These are availability counts, not independently verified correctness. The historical "VERIFIED" label is a heuristic macro/mapping check, not clinical or recipe validation. Machine-readable results, fingerprints, review templates and the recovery protocol accompany this repository under `research/`.

The September application uses Next.js 16, explicit 25–1,000 g portions in 25 g steps, and browser-only history. Only explicitly saved meals contribute to daily totals. Photos and health selections are not persisted in the journal. Recommendation inputs identify database nutrition per 100 g and do not include selected grams or saved history. Health badges reflect the returned context, not medical certification. Older screenshots depict historical interface states.

I. INTRODUCTION

Image-based food recognition is often evaluated as a visual classification or detection problem. In a diet-assistance setting, however, the predicted label is only an intermediate result. A class such as `palak_paneer`, `rice`, or `samosa` must be resolved to a nutrition entry before the system can report calories, protein, carbohydrates, or fat. An incorrect resolution can produce a recommendation that appears precise while being nutritionally wrong.

The problem is pronounced in Indian food images. A single meal may contain several dishes, and the same dish may appear under regional names, spelling variants, or dataset-specific labels. Plain rice, for example, may be recorded as `WhiteRice`, `satham`, or `rice`. A chutney label may refer to a side dish or to a small portion attached to a larger item. Visually similar dishes such as biryani and pulao may also differ in ingredient composition and macro values. These cases make direct string matching a weak basis for nutrition lookup.

This paper presents an implementation of the proposed food-recognition and nutrition-recommendation pipeline. The system uses YOLO11s for object detection, a curated Indian-food dataset, a SQLite nutrition database, and a recommendation layer that consumes already-computed nutrient values. The design separates three responsibilities: visual recognition, nutrition grounding, and recommendation generation. The detector identifies visible food items; the grounding layer decides whether a defensible database match exists; the recommendation module uses the resulting macro values and user profile to produce diet-aware guidance.

The contributions are:

1) a 72-class Indian-food object-detection dataset obtained by merging, normalizing, and visually checking five YOLO-format datasets;
2) a nutrition-grounding layer with reviewed matches, fallback matches, supplemental sources, and missing-nutrition states;
3) a FastAPI and Next.js prototype that links detection, macro calculation, serving adjustment, and personalized recommendations.

II. Related Work

Prior work relevant to this study can be grouped into Indian food recognition, nutrition database alignment, and diet recommendation systems. Methods for depth-based volume estimation and calibrated portion measurement are related, but they are outside the implemented scope because the present system reports database-backed per-100g macro values and leaves serving adjustment to the user.

Indian food recognition studies show that domain-specific data are necessary, but they do not fully address nutrition grounding. IndianFoodNet uses YOLO-style detection for Indian food items and is therefore closer to real meal photographs than single-label classification, where one image is forced into one category [1]. Khana provides a larger Indian cuisine benchmark and reports recurring difficulties such as class imbalance, regional variation, and visual similarity between dishes [2]. The Indian Thali study moves toward cluttered meal analysis through segmentation and controlled capture, although such capture assumptions are less convenient for ordinary web uploads [3]. Lightweight CNN and MobileNetV3 studies support compact architectures and augmentation for food recognition, but their experimental settings remain closer to classification than multi-item detection [4], [5]. These works motivate a detector trained on Indian food labels, but they leave the detector-to-nutrition link largely under-specified.

Nutrition database work addresses a different part of the problem. The Indian Food Composition Database study shows why cooked Indian foods require dedicated nutrient records rather than direct assumptions from raw ingredients [6]. The Synergistic Frameworks study identifies a practical compatibility gap: visual food labels and food-composition records often differ in spelling, language, or preparation name [7]. FKG.in demonstrates how language models and food knowledge graphs can parse regional recipe descriptions, but that setting begins with text rather than an image [8]. The present system therefore uses deterministic lookup, reviewed mappings, and recorded source metadata instead of delegating nutrient estimation to a language model.

Recent recommendation systems illustrate both the utility and the risk of generated diet advice. NutriGen and other web-based meal-planning systems generate personalized plans from structured user constraints [9], [10]. Diet Engine combines food detection with NLP-based dietary guidance and is close to the application direction of this work [11]. At the same time, evaluations of ChatGPT-style nutrient estimation from meal photographs show that well-formed responses do not guarantee accurate calorie or macro estimates [12]. The proposed design follows a conservative boundary: recommendations are generated after detection and macro calculation, while the numerical nutrition layer remains database-backed and inspectable.

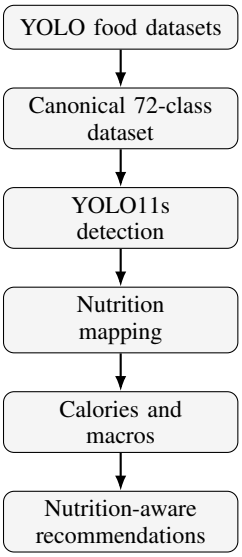

Fig. 1. End-to-end workflow for AI-based food recognition with nutrition-aware recommendations.

III. Proposed System

Fig. 1 summarizes the implemented system. The design uses a modular pipeline rather than a single end-to-end predictor because each stage has a different failure mode. Dataset normalization controls the detector label space, YOLO11s performs multi-item food detection, the nutrition layer resolves labels to database rows, and the recommendation module receives structured macro values. YOLO11s was selected as a compact detector suitable for a web prototype where inference time, model size, and multi-object localization are practical constraints. Recommendation text is generated only after the backend has calculated nutrition values.

Let a detector return

$$D_x = \{(\hat{c}_q, b_q, p_q)\}_{q=1}^{Q_x}, \tag{1}$$

where $\hat{c}_q$ is the predicted food class, b_q is the bounding box, and p_q is the confidence score. Let the nutrition database be

$$N = \{(n_j, \mathbf{m}_j, z_j)\}_{j=1}^{L}, \tag{2}$$

where n_j is a food name, $\mathbf{m}_j = [e_j, protein_j, carb_j, fat_j]$ is the nutrition vector, and z_j stores source metadata.

TABLE I
SOURCE DATASET POOL BEFORE PROJECT-SPECIFIC MERGING.

Source dataset	Classes	Train	Valid	Test
Indian food detection v1	17	7547	1053	453
Indian food v6	29	8088	213	105
Indian food v2 subset	7	1698	159	92
IndianFoodNet YOLO export	30	11387	1073	576
South Indian food detection v19	31	6478	1294	581

The mapping function M resolves a detector class to a nutrition row:

$$M(c) = \begin{cases} M_{\text{manual}}(c), & \text{if a reviewed semantic match exists,} \\ M_{\text{exact}}(c), & \text{if normalized names match,} \\ M_{\text{sup}}(c), & \text{if a verified supplemental row exists,} \\ M_{\text{fuzzy}}(c), & \text{if score}(c, n) \geq 0.78, \\ \emptyset, & \text{otherwise.} \end{cases} \tag{3}$$

The last case is a deliberate design choice. If no safe nutrition match exists, the system keeps the detection visible but does not assign macro values. For a diet-assistance interface, an explicit missing value is preferable to a confident but unsupported nutrient estimate.

For a meal image, the displayed macro vector is

$$\hat{\mathbf{m}}(x) = \sum_{q=1}^{Q_x} s_q \mathbf{m}_{M(\hat{c}_q)}, \tag{4}$$

where s_q is the serving multiplier selected by the user. Portion adjustment is therefore explicit. The system does not infer exact food mass from a single image without depth, calibration, or a reference object.

A. Runtime Data Contract

The backend response is structured so that the frontend does not infer missing or approximate values from presentation logic. Each detection carries the raw detector label, display name, confidence score, bounding box, image dimensions, macro values, nutrition unit, nutrition source, and missing-nutrition flag. If no valid food is detected, the API returns an explicit no-food state. If a detected class has no safe nutrition match, the detection remains visible while macro values remain unavailable. This contract keeps uncertainty visible to the user and prevents downstream recommendations from using hidden substitutions.

IV. DATASET CONSTRUCTION

Five source exports were merged into one label-normalized dataset [13], [14], [15], [16], [17]. Their names and class orders differ, so source labels were normalized for case, punctuation and selected semantic aliases before rewriting YOLO class indices. This engineering step preserves multi-food annotation rows but does not establish independent source families or resolve image-level reuse rights. The September audit qualifies all downstream detector claims.

TABLE II
REPRESENTATIVE GROUPS IN THE 72-CLASS INDIAN-FOOD LABEL SPACE.

Group	Example classes
Rice dishes	`rice`, `lemon_rice`, `sambar_satham`, `veg_briyani`
Breads	`roti`, `paratha`, `bhakri`, `bhatura`, `puri`
Curries	`chole`, `rajma_curry`, `fish_curry`, `palak_paneer`
Snacks	`samosa`, `onion_pakoda`, `medu_vada`, `khandvi`
Sweets	`gulab_jamun`, `ras_malai`, `kheer`, `jalebi`
Sides	`coconut_chutney`, `green_chutney`, `raita`, `salad`

TABLE III
GENERATED DATASET SPLIT SUMMARY.

Item	Result
Canonical food classes	72
Train image-label pairs	36,852
Validation image-label pairs	5,922
Held-out test image-label pairs	5,919
Total exported pairs	48,693
Classes with visual samples	72 / 72
Known annotation issues found	3

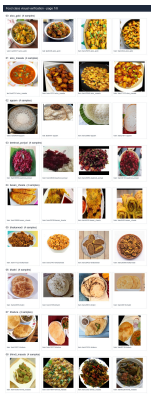

Fig. 2. Sample visual verification sheet generated from real exported dataset images.

The generated dataset used a 70/15/15 target ratio. Multi-label images were retained. Additional augmentation was restricted to training, but some source exports already contained augmented variants. The September audit found identical images spanning partitions; independence cannot be inferred from this splitting procedure.

A visual verification script sampled real images for every class and generated contact sheets. This check confirmed that all 72 classes had image samples after merging. It also found three local annotation issues: one `jalebi` sample that did not visually match jalebi, one `kaara_chutney` box placed on a vada, and one extremely thin `nandu_masala` box. These issues were retained as documented dataset defects rather than treated as class-order errors.

V. Detector Training and Evaluation

The final detector was trained with YOLO11s using 640-pixel images and batch size 16. Training resumed from epoch 56 and continued to epoch 100 on a Tesla T4 GPU. This configuration was chosen to keep the detector small enough for prototype inference while retaining multi-object localization, which is required for meal images containing more than one food item. The best logged validation mAP@50 was 0.83379 at epoch 78. On the held-out test split, the exported model reached 0.842 precision, 0.801 recall, 0.820 mAP@50, and 0.680 mAP@50:95 across 5,919 images and 8,875 instances.

TABLE IV
Final detector evaluation on validation and held-out test splits.

Split	Precision	Recall	mAP@50	mAP@50:95
Validation	0.847	0.807	0.830	0.692
Held-out test	0.842	0.801	0.820	0.680

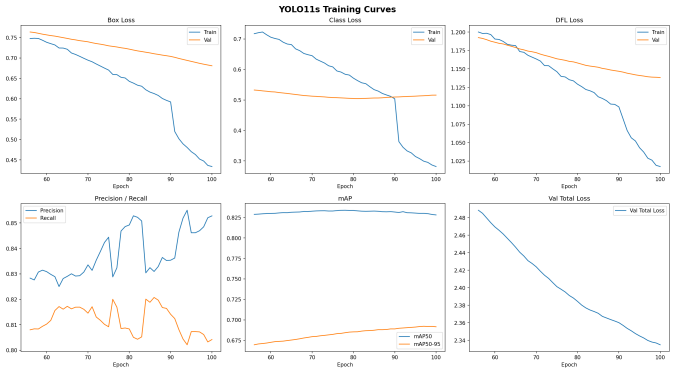

Fig. 3. YOLO11s training curves for the 72-class Indian-food detector.

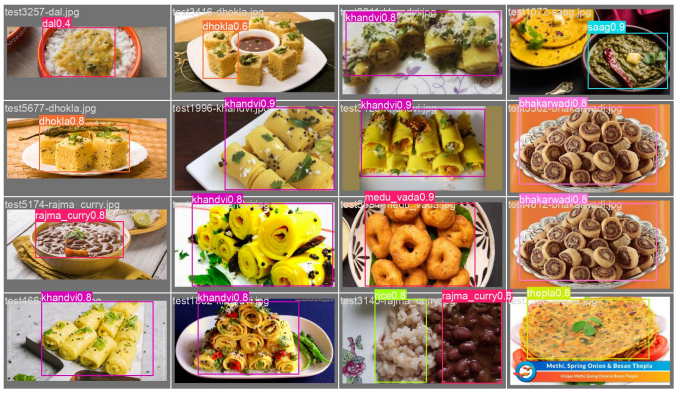

Fig. 4. Example held-out test predictions from the final YOLO11s detector.

The archived validation and test metrics are numerically close, but this does not rule out leakage. The confirmed split overlap prevents treating these numbers as proof of independent generalization. Confusion matrices and qualitative predictions remain descriptive historical artifacts; source-grouped evaluation and per-class analysis are required before stronger claims.

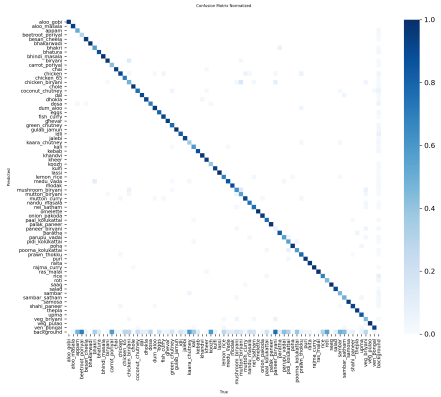

Fig. 5. Normalized confusion matrix from held-out test evaluation.

VI. Nutrition Mapping and Recommendations

The nutrition database was built from an Indian Nutrient Database spreadsheet and stored in SQLite [18]. The final table contains 1,010 food records with calories, protein, carbohydrates, fat, and source metadata. Five detector classes absent from the available INDB sheet were added as source-backed supplemental rows: `bhakarwadi`, `ghevar`, `jalebi`, `khandvi`, and `nandu_masala` [19], [20], [21], [22], [23].

TABLE V
Representative detector-to-nutrition mappings.

Detector class	Nutrition database row	Score
`aloo_gobi`	Potato cauliflower (Aloo gobhi)	1.00
`palak_paneer`	Spinach paneer (Palak paneer)	1.00
`rice`	Boiled rice (Uble chawal)	1.00
`chicken_biryani`	Chicken pulao	0.80
`ven_pongal`	Plain khitchdi	0.75

Table V shows that the mapping layer distinguishes exact semantic matches from preparation-family approximations. Direct semantic matches are treated as verified. Lower-score matches are retained only with review status, which prevents the interface from presenting all nutrition links as equally certain.

TABLE VI
Nutrition mapping verification.

Validation item	Result
Nutrition food records	1,010
YOLO-to-database mappings	103
Expanded classes checked	72
Expanded classes with mappings	72
Verified mappings	51
Review mappings with complete macros	21
Failed macro mappings	0
Expanded-class coverage	100.00%

The recommendation module receives detected foods, calculated calories, macro totals, user goal, calorie target, and health profile. It does not generate calorie or macronutrient values. This separation keeps numerical nutrition data under deterministic backend control while allowing the advice layer to operate on structured inputs.

The macro engine converts a calorie target into carbohydrate, protein, and fat grams. A selected goal defines the base macro split, and a health profile modifies that split before normalization. For a calorie target T and final macro percentages p_c, p_p, and p_f, the target grams are:

$$G_c = \frac{Tp_c}{4 \times 100}, \quad G_p = \frac{Tp_p}{4 \times 100}, \quad G_f = \frac{Tp_f}{9 \times 100}. \quad (5)$$

The recommendation service receives these calculated values along with the detected foods. For example, a high-carbohydrate fried meal under a weight-loss goal can trigger portion-control and protein-pairing advice, while a diabetic profile can shift the emphasis toward carbohydrate moderation and fiber. The output is therefore conditioned on database-backed macro values rather than on unverified nutrient estimates.

TABLE VII
BASE MACRO SPLITS USED BEFORE HEALTH-PROFILE ADJUSTMENT.

Goal	Carbs	Protein	Fat
Weight loss	40%	35%	25%
Muscle gain	40%	30%	30%
Maintenance	50%	25%	25%
Endurance	55%	20%	20%

VII. Prototype Validation

The prototype was implemented to verify that the trained detector, nutrition database, mapping logic, and recommendation interface could operate as one system. The backend uses FastAPI, PyTorch, Ultralytics YOLO, SQLite, and supporting Python libraries. The frontend uses Next.js, React, TypeScript, and Tailwind CSS. A user can upload an image, inspect detected boxes, review nutrition cards, adjust serving multipliers, and request dietary suggestions.

TABLE VIII
MAIN IMPLEMENTATION COMPONENTS AND RESPONSIBILITIES.

Component	Responsibility
Dataset builder	Parses source YOLO labels, normalizes class names, exports canonical splits
Vision service	Loads YOLO weights, runs inference, filters boxes, formats detections
Nutrition service	Loads SQLite rows, applies mappings, returns macros and provenance
Macro engine	Converts calorie targets and health profiles into macro goals
Recommendation service	Generates advice from detected foods and computed macros
Frontend interface	Handles upload, overlays, serving controls, macro display, and reset flow

The image-analysis response includes the detector label, display name, confidence, bounding box, image dimensions, macro values, nutrition source, and missing-nutrition flags. Runtime checks were run for classes that are easy to mishandle, including `aloo_gobi`, `palak_paneer`, `rice`, `AlooGobi`, and `WhiteRice`. These labels resolved to the expected nutrition rows. The API also returns an explicit no-food state when no valid detection is found.

Fig. 6. Prototype output showing detected foods with bounding boxes and nutrition-linked results.

TABLE IX
RUNTIME BEHAVIOR CHECKED DURING PROTOTYPE VALIDATION.

Condition	Expected behavior
`aloo_gobi`	Resolve to potato cauliflower nutrition row
`palak_paneer`	Resolve to spinach paneer nutrition row
`WhiteRice`	Normalize legacy label to boiled rice
No detected food	Return empty detections and no-food flag
Unmapped class	Preserve detection and mark nutrition unavailable

VIII. Validation Scope

Validation was performed at several levels because the user-facing nutrition result depends on more than detector accuracy alone. Dataset checks confirmed class folders, label files, split counts, and visual samples. Model evaluation measured detection performance on validation and held-out test splits. Nutrition checks verified that detector classes resolve to complete macro rows. Runtime checks confirmed that the backend response contains the fields required for overlays, nutrition display, serving controls, and recommendation prompts.

TABLE X
VALIDATION LAYERS USED IN THE PROJECT.

Layer	What was checked
Dataset	Split counts, class coverage, visual samples, annotation issues
Detector	Precision, recall, mAP@50, mAP@50:95, prediction examples
Nutrition	Database rows, mapping coverage, macro completeness, sources
API	Upload validation, no-food state, mapped and unmapped detections
Frontend	Overlay rendering, nutrition cards, serving controls, recommendations

The layered validation separates failure modes that would otherwise be hidden inside a single demonstration. A detector can classify a food correctly while the nutrition service maps it poorly. A nutrition database can contain complete rows while the frontend receives fields under unexpected names. Reporting these checks separately makes the system easier to audit and clarifies what has, and has not, been validated.

IX. DISCUSSION

The results show that nutrition-aware food recognition cannot be evaluated by detector mAP alone. A visually correct prediction can still produce an incorrect dietary result if it is mapped to the wrong food-composition row. The mapping table, source metadata, and explicit missing-value behavior are therefore part of the technical pipeline, not only backend implementation details.

The system also reflects a practical trade-off. It does not attempt exact portion estimation from a single image, because that would require assumptions that are difficult to verify in ordinary user photographs. Instead, it reports database-backed macro values and lets the user adjust serving multipliers. This limits automation, but it keeps the reported nutrition values closer to the evidence available to the system.

X. LIMITATIONS

The system does not estimate exact portion mass from the image. Nutrition values are reported from database rows and adjusted through user-selected serving multipliers. More reliable portion estimation would require segmentation, depth cues, reference objects, or calibrated serving-size assumptions. The detector may also show uneven per-class behavior for visually similar foods and classes with fewer diverse samples. Finally, the 21 review mappings should be interpreted as practical approximations rather than exact recipe equivalence.

XI. CONCLUSION

This paper presented *AI-Based Food Recognition with Nutrition-Aware Recommendations*, a system for Indian food detection, nutrition lookup, macro calculation, and personalized dietary guidance. The work combines a YOLO11s detector, a 72-class canonical Indian-food dataset, a conservative nutrition-mapping layer, and a full-stack prototype. The detector achieved 0.820 mAP@50 on the held-out test split, and the nutrition layer mapped all 72 expanded classes with zero failed macro mappings. The study indicates that food recognition becomes more useful for dietary support when visual predictions are connected to auditable nutrition data and passed to the recommendation layer with uncertainty preserved.

ACKNOWLEDGMENT

The implementation and dataset engineering artifacts used in this manuscript were developed as part of a final year project in the Department of Computer Science and Engineering, National Institute of Technology Srinagar. The authors acknowledge the technical supervision and review provided by project guide Dr. Lavanya Madhuri Bollipo and project co-guide Dr. Insha Majeed.

REFERENCES

[1] R. Agarwal, T. Choudhury, N. J. Ahuja, and T. Sarkar, "IndianFoodNet: Detecting indian food items using deep learning," *International Journal of Computational Methods and Experimental Measurements*, vol. 11, no. 4, pp. 221–232, 2023.

[2] O. Prabhu, "Khana: A comprehensive Indian cuisine dataset," *arXiv preprint arXiv:2509.06006*, 2025.

[3] Y. Arora, A. Arun, and C. V. Jawahar, "What is there in an Indian Thali?" in *Proceedings of the 16th Indian Conference on Computer Vision, Graphics and Image Processing*, 2025.

[4] R. U. Haque, R. H. Khan, A. S. M. Shihavuddin, M. M. M. Syeed, and M. F. Uddin, "Lightweight and parameter-optimized real-time food calorie estimation from images using CNN-based approach," *Applied Sciences*, vol. 12, no. 19, p. 9733, 2022.

[5] J. Patel and K. Modi, "Indian food image classification and recognition with transfer learning technique using MobileNetV3 and data augmentation," in *Engineering Proceedings*, vol. 56, no. 1, 2023, p. 197.

[6] A. Vijayakumar, H. B. Dubasi, A. Awasthi, and L. M. Jaacks, "Development of an Indian food composition database," *Current Developments in Nutrition*, vol. 8, no. 7, p. 103790, 2024.

[7] "Synergistic frameworks for Indian food computing: An analysis of nutritional and visual dataset compatibility," 2025.

[8] S. K. Gupta, L. Dey, P. P. Das, G. Trilok-Kumar, and R. Jain, "Enhancing FKG.in: Automating Indian food composition analysis," *arXiv preprint arXiv:2412.05248*, 2024.

[9] S. Khamesian, A. Arefeen, S. M. Carpenter, and H. Ghasemzadeh, "NutriGen: Personalized meal plan generator leveraging large language models to enhance dietary and nutritional adherence," *arXiv preprint arXiv:2502.20601*, 2025.

[10] S. Hussain, N. Khan, S. A. A. Shah, S. Bano, and A. W. Khan, "AI-driven personalized meal planning: A web-based platform for tailored nutrition and health management," *International Journal of Computer Applications*, vol. 187, no. 57, pp. 17–29, 2025.

[11] A. M. Saad, M. R. H. Rahi, M. M. Islam, and G. Rabbani, "Diet engine: A real-time food nutrition assistant system for personalized dietary guidance," *Food Chemistry Advances*, vol. 7, p. 100978, 2025.

[12] C. O'Hara, G. Kent, A. C. Flynn, E. R. Gibney, and C. M. Timon, "An evaluation of ChatGPT for nutrient content estimation from meal photographs," *Nutrients*, vol. 17, no. 4, p. 607, 2025.

[13] project-z0tql, "Indian food detection, version 1," https://universe.roboflow.com/project-z0tql/indian-food-detection-kzw9g/dataset/1, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.

[14] microplastics-vypxl, "Indian food, version 6," https://universe.roboflow.com/microplastics-vypxl/indian-food-txofy/dataset/6, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.

[15] indianfood, "Indianfood-7, version 2," https://universe.roboflow.com/indianfood/indian_food-pwzlc/dataset/2, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.

[16] IndianFoodNet contributors, "Indianfoodnet-30, version 1," https://universe.roboflow.com/indianfoodnet/indianfoodnet/dataset/1, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.

[17] bharani-1ucid, "South indian food detection, version 19," https://universe.roboflow.com/bharani-1ucid/south-indian-food-detection/dataset/19, source/version recovered from local dataset export metadata, inspected 2026-09-12; image-level rights require separate review.
[18] Indian Nutrient Database, "Indian nutrient database spreadsheet used in project data layer," 2026, pLACEHOLDER BIBTEX ENTRY: replace with the official publication or institutional citation for INDB.xlsx before final submission.
[19] FatSecret India, "Evolve bhakarwadi nutrition facts per 100 g," Online nutrition record, 2026, accessed 2026-06-06.
[20] Clearcals, "Ghevar recipe calories and nutrition per 100 g," Online nutrition record, 2026, accessed 2026-06-06.
[21] ——, "Jalebi recipe calories and nutrition per 100 g," Online nutrition record, 2026, accessed 2026-06-06.
[22] ——, "Khandvi recipe calories and nutrition per 100 g," Online nutrition record, 2026, accessed 2026-06-06.
[23] ——, "Nandu kari recipe calories and nutrition per 100 g," Online nutrition record, 2026, accessed 2026-06-06.